

Coded AI Agent

Procurement AI Agent — Automated PO and RFQ Drafting

What is an AI agent?

An AI agent is an autonomous software system powered by a Large Language Model (LLM) that processes complex tasks, and takes decisive actions using connected tools or APIs to achieve specific goals with minimal human intervention.

Step 1: Prerequisites

1. Python

Python is a programming language that lets you work quickly and integrate systems more effectively.

Download and install python from <https://www.python.org/downloads/>

Be sure to check all checkboxes during installation.



2. Ollama

Ollama is the easiest way to automate your work using open models, while keeping your data safe.

Download and install ollama from <https://ollama.com/download>



qwen3.5

↓ 15.4M Downloads Updated 1 month ago

Qwen 3.5 is a family of open-source multimodal models that delivers exceptional utility and performance.

[vision](#) [tools](#) [thinking](#) [cloud](#) [0.8b](#) [2b](#) [4b](#) [9b](#) [27b](#) [35b](#) [122b](#)

[CLI](#) [cURL](#) [Python](#) [JavaScript](#)

```
ollama run qwen3.5
```



3. Language model

Download a Large Language Model like qwen3.5. This will be the main driving force for the agent.

In order to do so, open a terminal / command prompt window and paste the following:

```
ollama pull qwen3.5
```

```
(c) Microsoft Corporation. All rights reserved.  
C:\Users\>ollama pull qwen3.5
```

4. Libraries

Download and install the required libraries by pasting the following in the terminal:

```
pip install streamlit ollama python-docx
```

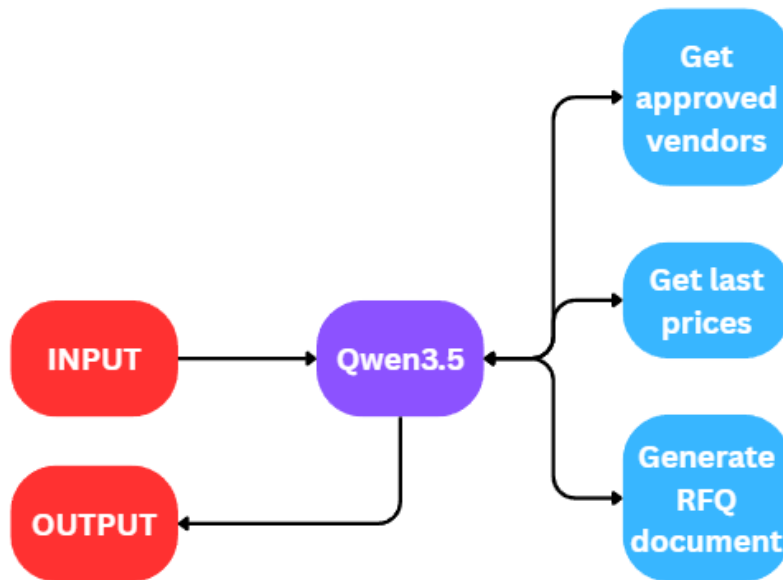
5. SQLite database

SQLite is a free and open-source relational database engine. We require a database including tables for vendor information, purchase order history and benchmark rates for goods.

Step 2: Understanding what the application does

An AI agent that accepts plain English procurement instructions and autonomously:

1. Queries the local SQLite vendor database.
2. Retrieves historical purchase prices and market benchmarks.
3. Generates a formatted Word RFQ document.



Step 3: Setting up application structure

1. Create a folder with the SQLite database(Eg. UC_P04_procurement.db) inside.
2. Create a python file (Eg. UC_P04_agent.py).
3. Open the python file in a text editor.

Step 4: Writing the application

Here, we'll follow a stepwise process to write each part of the application using prompts to an LLM which would generally give us the code we're looking for:

1. IMPORT BLOCK

Import os, BytesIO from io, date from datetime, connect from sqlite3, streamlit, chat from ollama, and Document from python-docx.

Create a path DB_PATH to the SQLite database named "UC_P04_procurement.db".

Create a path OUT_DIR to the directory in which this file is saved.

2. TOOL 1: GET APPROVED VENDORS

Write a tool function called `get_approved_vendors(category: str)` that queries a SQLite table 'vendors' for rows where category matches the input (using LIKE).

Return a formatted string with vendor name, country, rating, and lead_days.

Handle no-results case.

3. TOOL 2: GET LAST PURCHASE PRICES

Write a tool function called `get_last_prices(item: str)` that queries a SQLite table 'historical_pos' joined with 'vendors' for an item matching the input (LIKE).

Return the last 3 prices with dates and vendor names, the average of those 3, and the current market benchmark from 'benchmark_rates'.

4. TOOL 3: GENERATE RFQ DOCUMENT

Write a tool function called `generate_rfq_docx(details: str)` that parses a pipe-delimited string 'item|qty|unit|delivery_date|vendor1,vendor2'.

Use python-docx to create a Word document with: title, RFQ number, date, vendor list, item details table, and NALCO standard T&C bullets.

Save the file with a timestamp filename. Return the filename.

5. DOWNLOAD BUTTON

Write a Streamlit function `generate_docx_download_button()` that searches `OUT_DIR` for the most recently modified `RFQ_*.docx` file, loads it into a `BytesIO` object, displays an `st.download_button()` using the original filename and DOCX MIME type, then deletes the original file after creating the download button.

Return only the function.

6. STREAMLIT PAGE SETUP AND AGENT INITIALISATION

Set up a Streamlit page with title 'NALCO Procurement AI Agent' and caption "Ollama + SQLite — 100% local, zero external API".

Register three tools in a list along with a dictionary of tool names.

7. USER INTERFACE AND AGENT INVOCATION

Add two example instructions as markdown hints.

Create a Streamlit `text_input` for user instructions. Setup system prompt from `'https://smith.langchain.com/hub/hwchase17/react'`.

Setup message structure with system prompt for ollama chat.

Add a primary button 'Run Agent'. On click, invoke an agent loop with ollama chat using the model 'qwen3.5', tools and the user input messages, running till there are no tool calls to make.

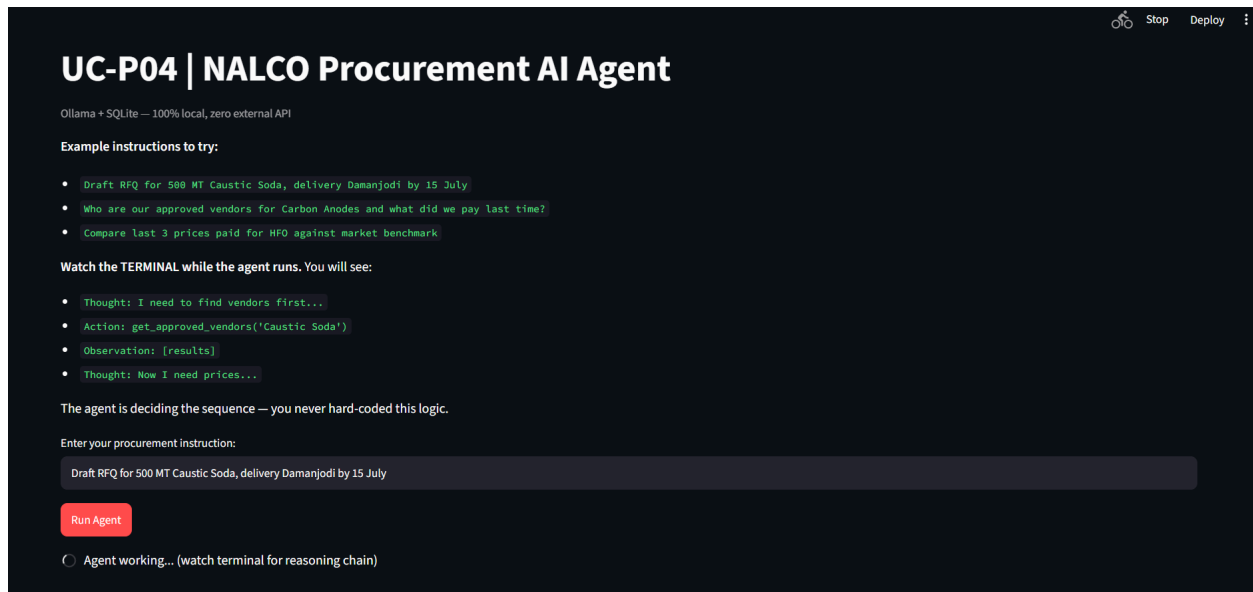
Display the output, check for any .docx files created recently, and add a download button for the latest one.

Handle execution errors.

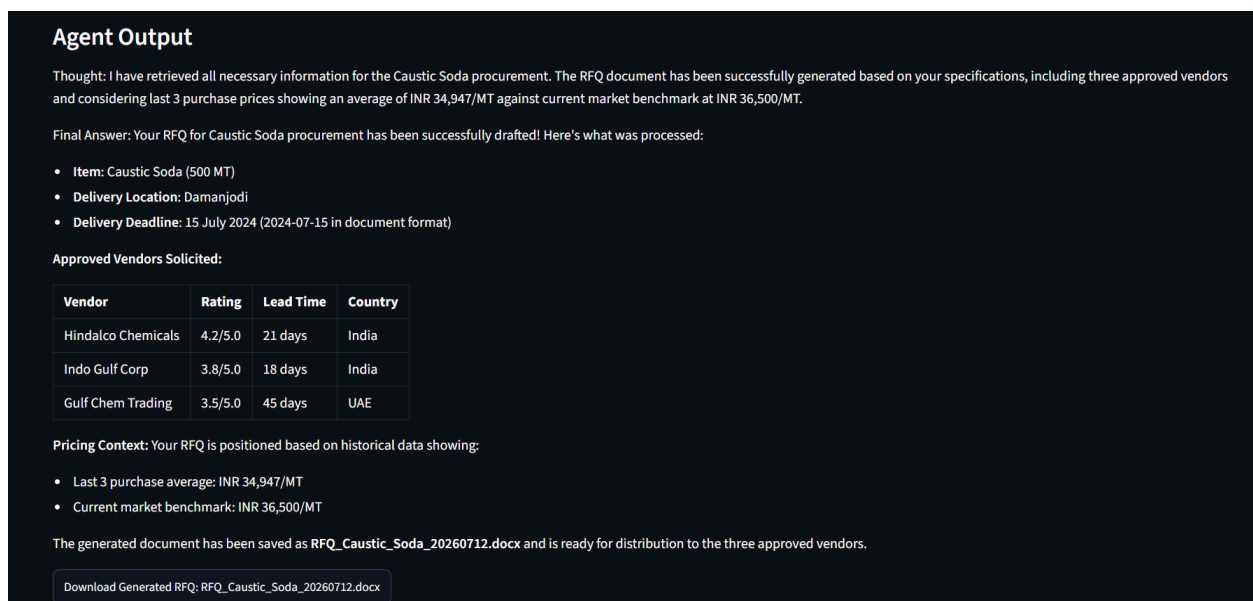
You may need to add code or make edits to avoid errors.

Step 5: Running the application

1. Open a **terminal**.
2. Navigate to the directory where your files are stored by typing '`cd`' followed by the address of the directory.
3. Enter **`streamlit run UC_P04_agent.py`** OR **`python -m streamlit run UC_P04_agent.py`**
4. You are presented with a **Streamlit** web app with some example prompts followed by a text input and a '**run agent**' button.
5. Enter a **prompt** of your choice and run the agent.



6. The results are displayed in the web app. You can see the reasoning process and **tools called** in the terminal.



```

Thinking: The user wants me to draft an RFQ (Request for Quotation) for 500 MT of Caustic Soda with delivery deadline of 15th July at Damanjodi.

To generate a proper RFQ document, I need:
1. Approved vendors for the category "Caustic Soda"
2. Recent purchase prices and market benchmark rates for Caustic Soda

Let me first get the approved vendors for Caustic Soda, then get the last prices to understand what NALCO has paid recently. After that I can generate the RFQ document.

I should start by getting both pieces of information in parallel since they're independent operations needed before generating the final RFQ docx.
Content:
Calling get_approved_vendors with arguments {'category': 'Caustic Soda'}
Result: Approved vendors for Caustic Soda:
Vendor: Hindalco Chemicals | Country: India | Rating: 4.2/5.0 | Lead Time: 21 days
Vendor: Indo Gulf Corp | Country: India | Rating: 3.8/5.0 | Lead Time: 18 days
Vendor: Gulf Chem Trading | Country: UAE | Rating: 3.5/5.0 | Lead Time: 45 days
Calling get_last_prices with arguments {'item': 'Caustic Soda'}
Result: Last 3 purchase prices for Caustic Soda:
2024-11-02 | INR 31,681/unit | Vendor: Gulf Chem Trading
2024-10-18 | INR 37,645/unit | Vendor: Gulf Chem Trading
2024-09-15 | INR 35,515/unit | Vendor: Indo Gulf Corp
Average of last 3: INR 34,947
Current market benchmark: INR 36,500/MT
vs benchmark: INR 1,553 below market rate
Thinking: Now I have all the information needed to generate an RFQ document. Let me format it properly according to the tool's required input format:

'details': 'item|quantity|unit|delivery_date|vendor1,vendor2,vendor3'

So for this case:
- item: Caustic Soda
- quantity: 500 MT (need to be careful about units - it shows both "MT" in the prompt and prices are shown as "/MT" so I'll use MT)
- unit: MT

```

Alternate use cases

The agent can be used to achieve any task so long as you have the required **tools** with good documentation.

For example, generating a list of all suppliers based in a particular country requires a tool function that checks the ‘vendors’ table for entries with the country field as input argument.

A model compatible with tool-calling can pass the required arguments when calling the function.

To help the LLM pick the correct tool, it is recommended to add **input** and **output hinting** to each tool function along with a well-defined **docstring**.